МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение

высшего образования

«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии

Департамент цифровых, робототехнических систем и электроники

Отчет к лабораторной работе № 7

Тема: Структурные паттерны: Заместитель

Дисциплина: **«Паттерны проектирования»**

Выполнил:

Студент группы ПИЖ-б-о-23-1
направление подготовки
09.03.04 «Программная
инженерия»

Постников Илья Викторович

Проверил:

Доцент ДЦРСИЭ

Альбекова З. М.

Ставрополь 2026

Цель: изучить принцип работы поведенческих паттернов проектирования «Команда» (Command) и «Итератор» (Iterator) и реализовать прикладную задачу «Графический пароль» с использованием данных паттернов.

Задание: необходимо разработать приложение для работы с графическим паролем и реализовать два поведенческих паттерна из списка: «Команда», «Итератор».

В соответствии с заданием были выбраны паттерны «Команда» (Command) и «Итератор» (Iterator).

Вариант 8 - Графический пароль

Разработанное приложение позволяет формировать, сохранять и проверять графический пароль, а также демонстрирует работу поведенческих паттернов: команда используется для инкапсуляции действий пользователя с возможностью отмены и повтора, а итератор - для последовательного обхода коллекции шаблонов.

1. Описание паттерна «Команда»

Паттерн «Команда» (Command) - это поведенческий паттерн проектирования, который инкапсулирует запрос в виде объекта, позволяя параметризовать клиентов командами, ставить команды в очередь, логировать их и поддерживать отмену операций.

Данный паттерн применяется в ситуациях, когда нужно отделить инициатора действия от его исполнителя, а также когда требуется поддержка истории действий и отмены.

В разработанном приложении паттерн «Команда» используется для инкапсуляции действий пользователя при формировании графического пароля. В программе определены следующие роли:

- Command - абстрактная команда с методами execute() и undo();
- SelectCellCommand - команда выбора ячейки;
- ClearSelectionCommand - команда очистки последовательности;
- ApplyPatternCommand - команда применения шаблона;

- CommandInvoker - инициатор, который выполняет команды и хранит историю для отмены и повтора.

Все действия пользователя проходят через CommandInvoker:

self.invoker.execute(SelectCellCommand(self, num))

Отмена и повтор реализованы в CommandInvoker через два стека - историю выполненных команд и стек повторов. Кнопки «↶ Отменить» и «↷ Повторить» позволяют пользователю откатывать действия.

Таким образом, клиент не знает, как именно выполняется действие, - он только формирует команду и передаёт её инициатору.

2. Описание паттерна «Итератор»

Паттерн «Итератор» (Iterator) - это поведенческий паттерн проектирования, который предоставляет способ последовательного обхода элементов коллекции без раскрытия её внутреннего представления.

Данный паттерн применяется в ситуациях, когда нужно обходить коллекцию, не завися от её внутренней структуры, или когда требуется несколько вариантов обхода.

В разработанном приложении паттерн «Итератор» используется для последовательного обхода коллекции шаблонов графического пароля. В программе определены следующие роли:

- PatternIterator - абстрактный итератор с методами has_next(), next(), has_previous(), previous(), current(), index(), total();
- PatternCollectionIterator - конкретный итератор;
- PatternCollection - агрегат, создающий итератор через create_iterator().

Навигация по шаблонам реализована кнопками ◄ и ►. Под ними отображается текущий шаблон и его позиция (например, «Крест (1/3)»).

Кнопка «Применить шаблон» применяет текущий шаблон через команду ApplyPatternCommand.

Клиент (AppWindow) работает только с интерфейсом PatternIterator, не зная, как хранятся шаблоны внутри PatternCollection.

3. Описание разработанного приложения

Разработанное приложение предназначено для формирования, сохранения и проверки графического пароля.

Приложение реализовано на языке Python с использованием библиотеки Tkinter.

Основное окно содержит сетку из девяти скруглённых ячеек, пронумерованных от 1 до 9. Пользователь формирует пароль путём последовательного выбора ячеек.

К паролю предъявляются требования:

- длина последовательности от 4 до 9 ячеек;
- одна и та же ячейка не может быть выбрана повторно;
- созданный пароль сохраняется в виде SHA-256 хэша.
- Приложение содержит два режима: «Создание пароля» и «Вход».

Дополнительно реализованы:

- кнопки «↶ Отменить» и «↷ Повторить» (паттерн «Команда»);
- навигация по шаблонам через ◄ / ► с отображением позиции (паттерн «Итератор»);
- кнопка «Применить шаблон»;
- кнопка «Очистить»;
- кнопка «Сбросить сохранённый пароль».

4. Диаграмма классов

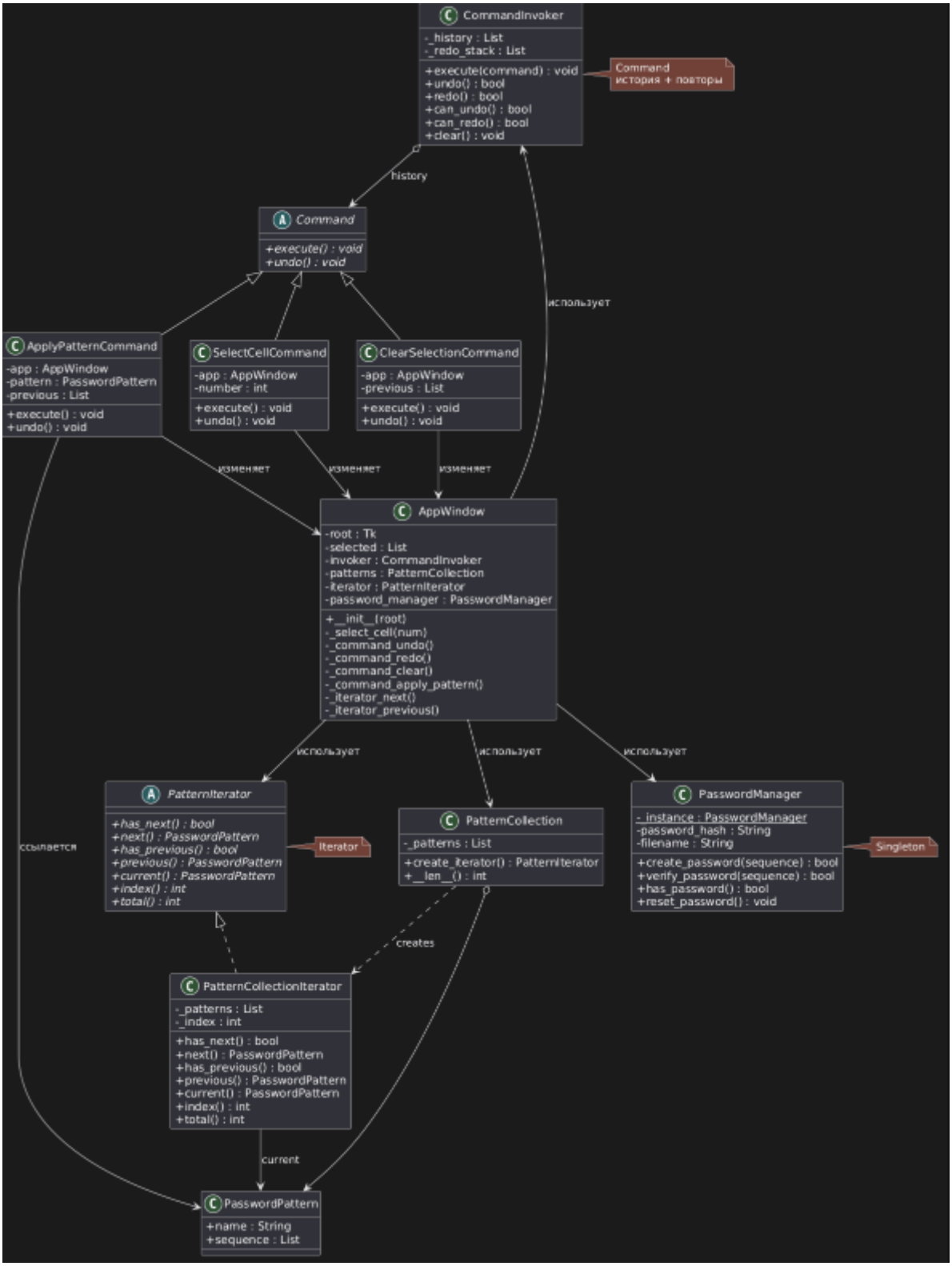


Рисунок 1 - Диаграмма классов приложения «Графический пароль»

Диаграмма содержит:

- Command, SelectCellCommand, ClearSelectionCommand, ApplyPatternCommand, CommandInvoker - паттерн «Команда»;
- PatternIterator, PatternCollectionIterator, PatternCollection - паттерн «Итератор»;
- AppWindow, PasswordManager, PasswordPattern - основные классы приложения.

5. Реализация паттерна «Команда»

Базовый интерфейс:

```
class Command(ABC):
    @abstractmethod
    def execute(self):
        pass

    @abstractmethod
    def undo(self):
        pass
```

Конкретная команда выбора ячейки:

```
class SelectCellCommand(Command):
    def __init__(self, app, number):
        self.app = app
        self.number = number

    def execute(self):
        self.app.selected.append(self.number)

    def undo(self):
        if self.number in self.app.selected:
            self.app.selected.remove(self.number)
```

Команда очистки с сохранением предыдущего состояния:

```
class ClearSelectionCommand(Command):
    def __init__(self, app):
        self.app = app
        self.previous = None

    def execute(self):
        self.previous = list(self.app.selected)
        self.app.selected = []

    def undo(self):
        if self.previous is not None:
            self.app.selected = list(self.previous)
```

Команда применения шаблона:

```
class ApplyPatternCommand(Command):
    def __init__(self, app, pattern):
        self.app = app
        self.pattern = pattern
        self.previous = None

    def execute(self):
        self.previous = list(self.app.selected)
        self.app.selected = list(self.pattern.sequence)

    def undo(self):
        if self.previous is not None:
            self.app.selected = list(self.previous)
```

Инициатор:

```
class CommandInvoker:
    def __init__(self):
        self._history = []
```

```
    self._redo_stack = []

def execute(self, command):
    command.execute()
    self._history.append(command)
    self._redo_stack.clear()

def undo(self):
    if not self._history:
        return False
    cmd = self._history.pop()
    cmd.undo()
    self._redo_stack.append(cmd)
    return True

def redo(self):
    if not self._redo_stack:
        return False
    cmd = self._redo_stack.pop()
    cmd.execute()
    self._history.append(cmd)
    return True

def can_undo(self):
    return len(self._history) > 0

def can_redo(self):
    return len(self._redo_stack) > 0

def clear(self):
```

```
        self._history.clear()
        self._redo_stack.clear()
```

В AppWindow все действия пользователя проходят через инициатор:

```
def _select_cell(self, num):
    ...
    self.invoker.execute(SelectCellCommand(self, num))
    self._update_ui()

def _command_undo(self):
    if self.invoker.undo():
        self._update_ui()

def _command_redo(self):
    if self.invoker.redo():
        self._update_ui()
```

6. Реализация паттерна «Итератор»

Абстрактный итератор:

```
class PatternIterator(ABC):
    @abstractmethod
    def has_next(self): pass

    @abstractmethod
    def next(self): pass

    @abstractmethod
    def has_previous(self): pass

    @abstractmethod
    def previous(self): pass
```

```
    @abstractmethod
    def current(self): pass

    @abstractmethod
    def index(self): pass

    @abstractmethod
    def total(self): pass
```

Конкретный итератор:

```
class PatternCollectionIterator(PatternIterator):
    def __init__(self, patterns):
        self._patterns = list(patterns)
        self._index = 0

    def has_next(self):
        return self._index < len(self._patterns) - 1

    def next(self):
        if self.has_next():
            self._index += 1
        return self.current()

    def has_previous(self):
        return self._index > 0

    def previous(self):
        if self.has_previous():
            self._index -= 1
        return self.current()
```

```
    def current(self):
        if not self._patterns:
            return None
        return self._patterns[self._index]

    def index(self):
        return self._index + 1 if self._patterns else 0

    def total(self):
        return len(self._patterns)
```

Агрегат:

```
class PatternCollection:
    def __init__(self, patterns):
        self._patterns = list(patterns)

    def create_iterator(self):
        return PatternCollectionIterator(self._patterns)

    def __len__(self):
        return len(self._patterns)
```

Использование в AppWindow:

```
self.patterns = PatternCollection([...])
self.iterator = self.patterns.create_iterator()

def _iterator_next(self):
    self.iterator.next()
    self._update_iterator_label()
```

7. Результаты работы программы

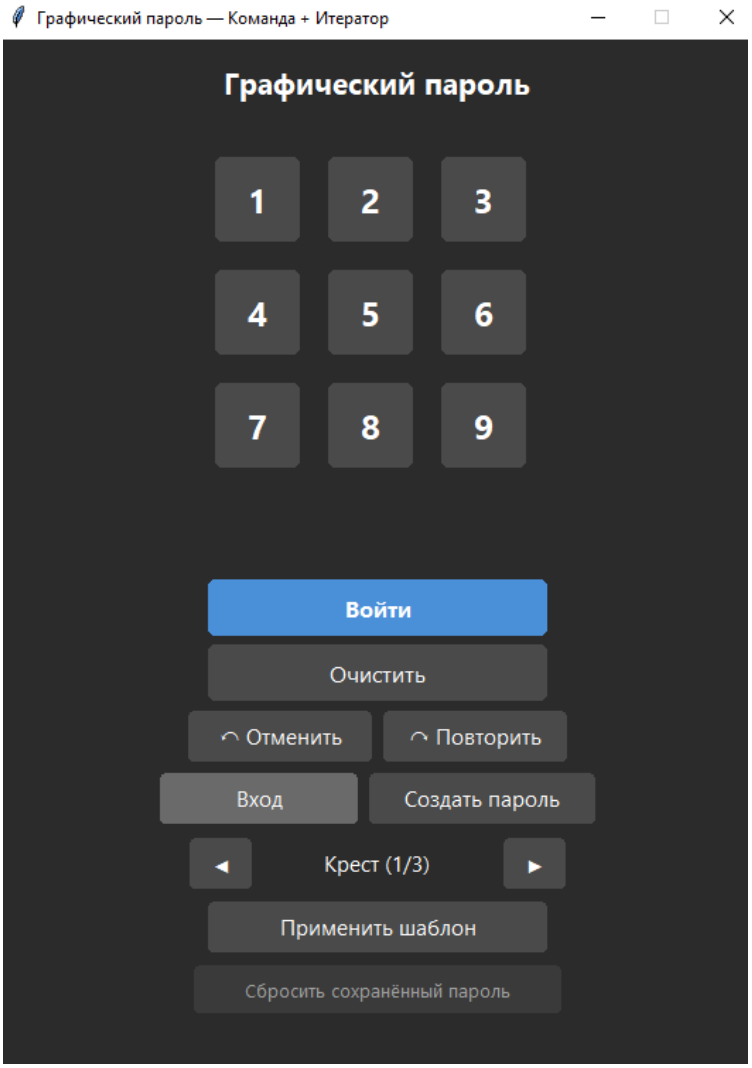


Рисунок 2 - Главное окно приложения

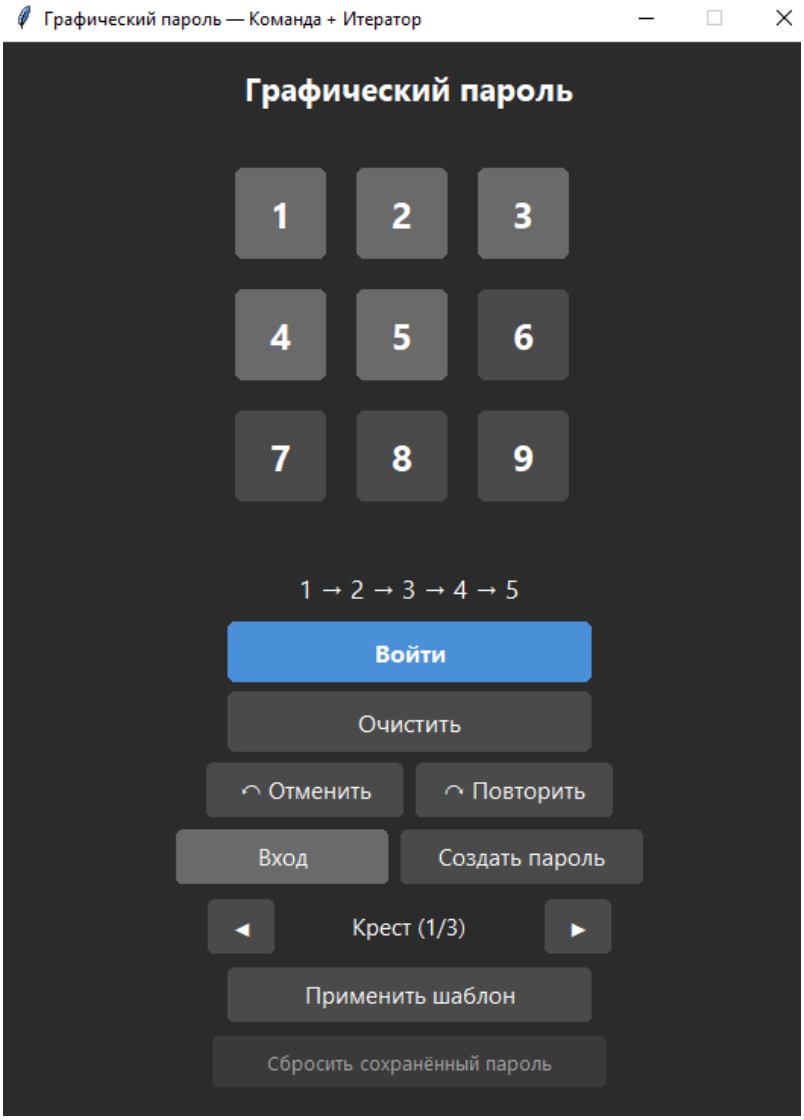


Рисунок 3 - Формирование последовательности (команды выбора ячейки)

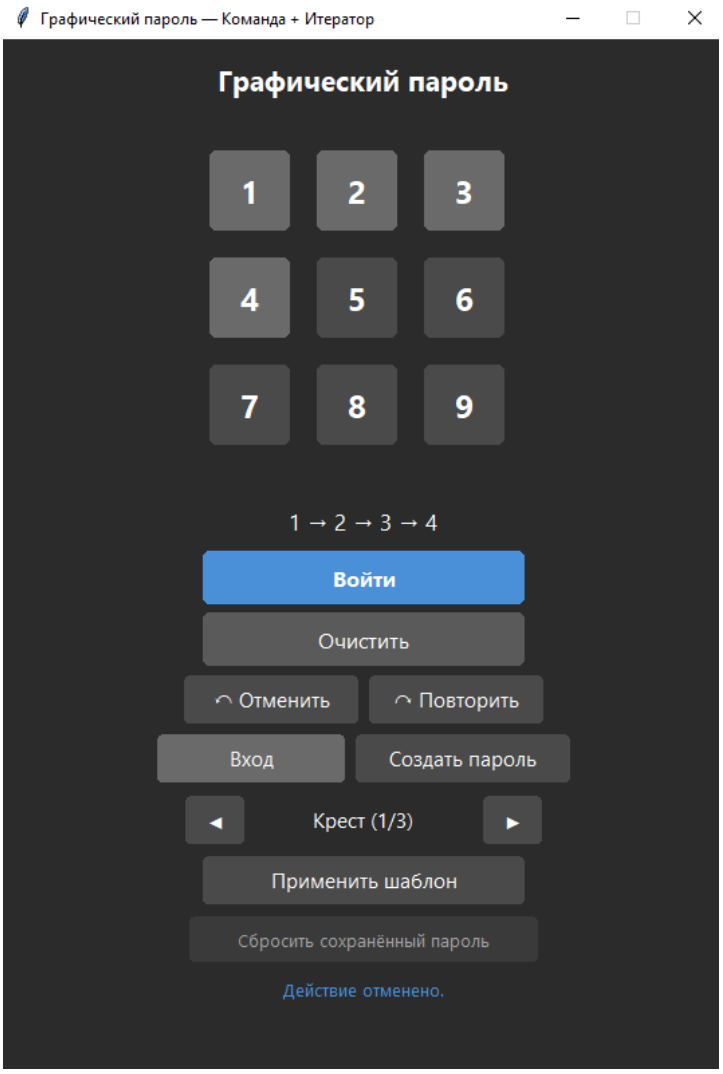


Рисунок 4 - Отмена последнего действия (↶ Отменить)

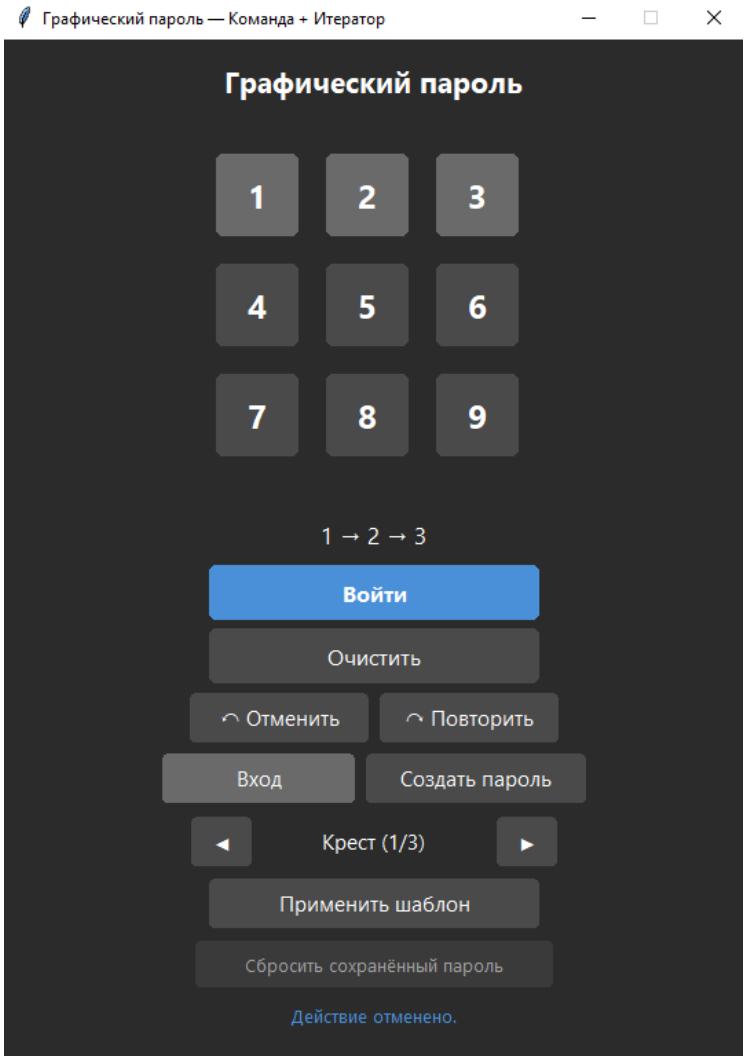


Рисунок 5 - Повтор отменённого действия (↷ Повторить)

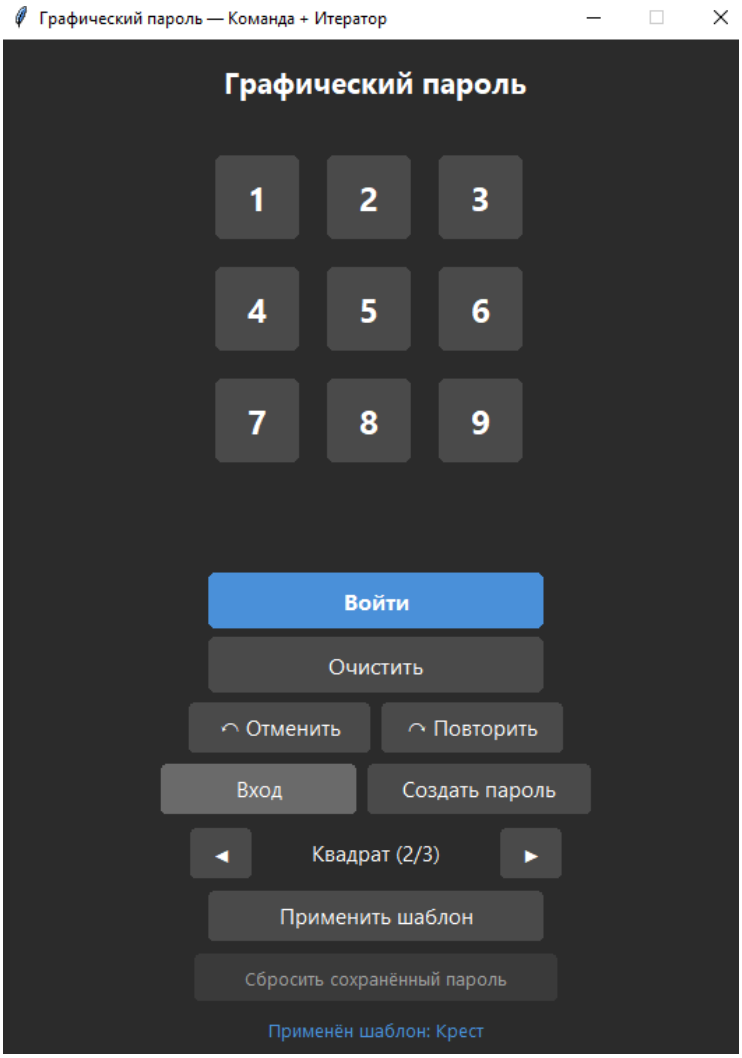


Рисунок 6 - Навигация по шаблонам (итератор, 1/3 → 2/3)

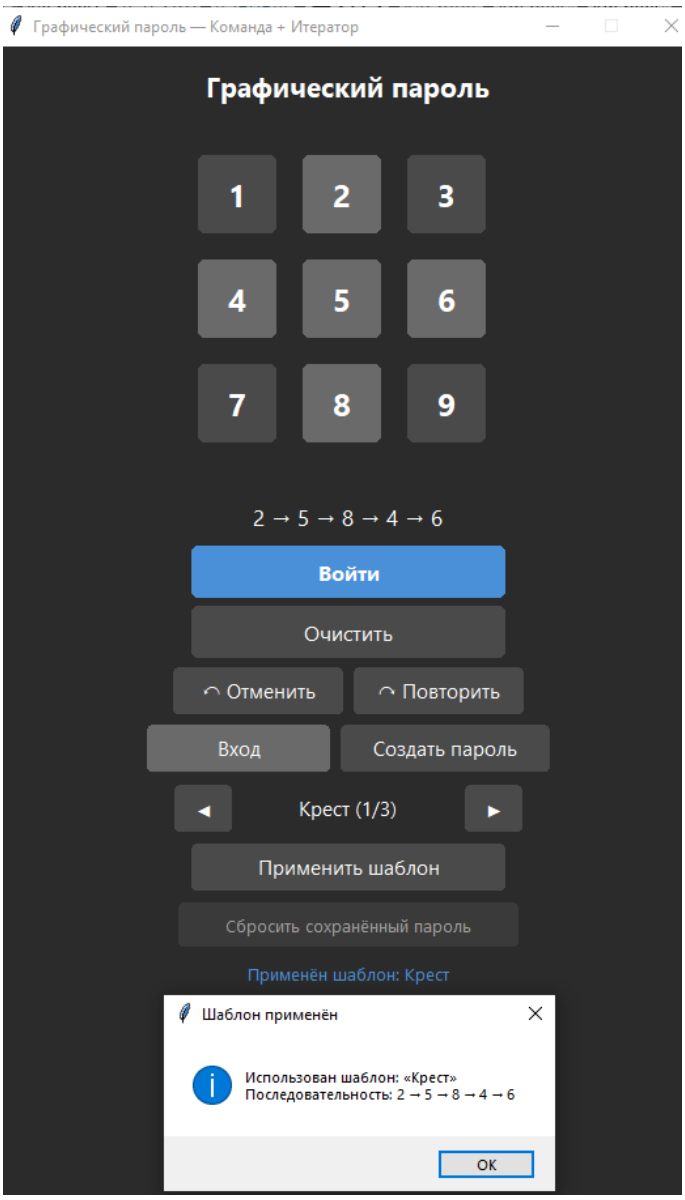


Рисунок 7 - Применение шаблона через команду

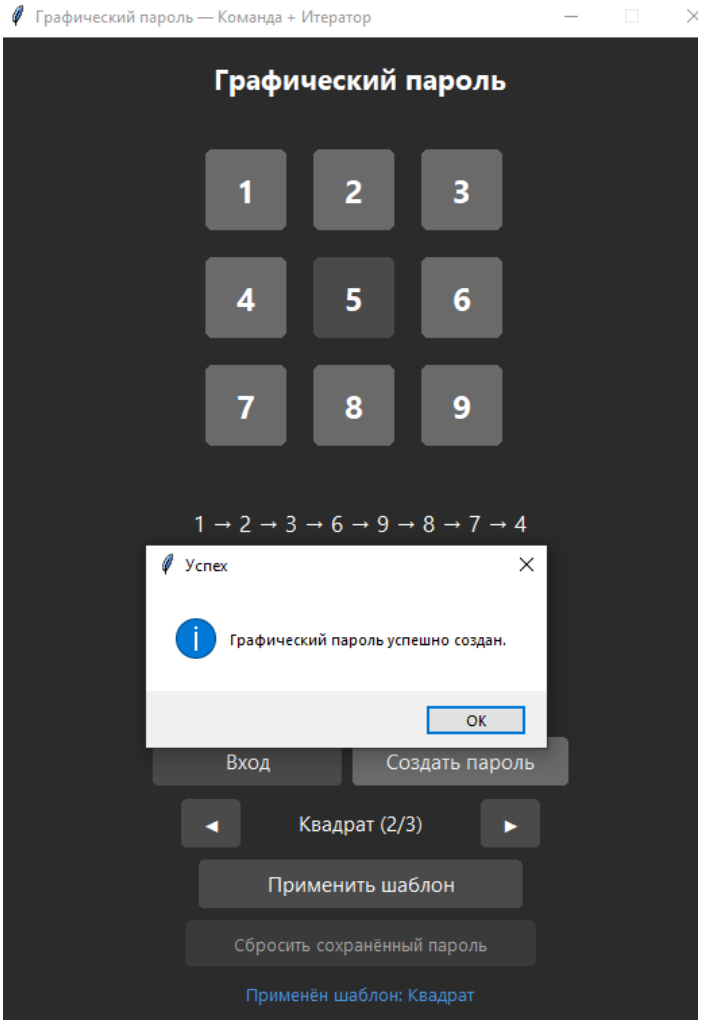


Рисунок 8 - Создание пароля

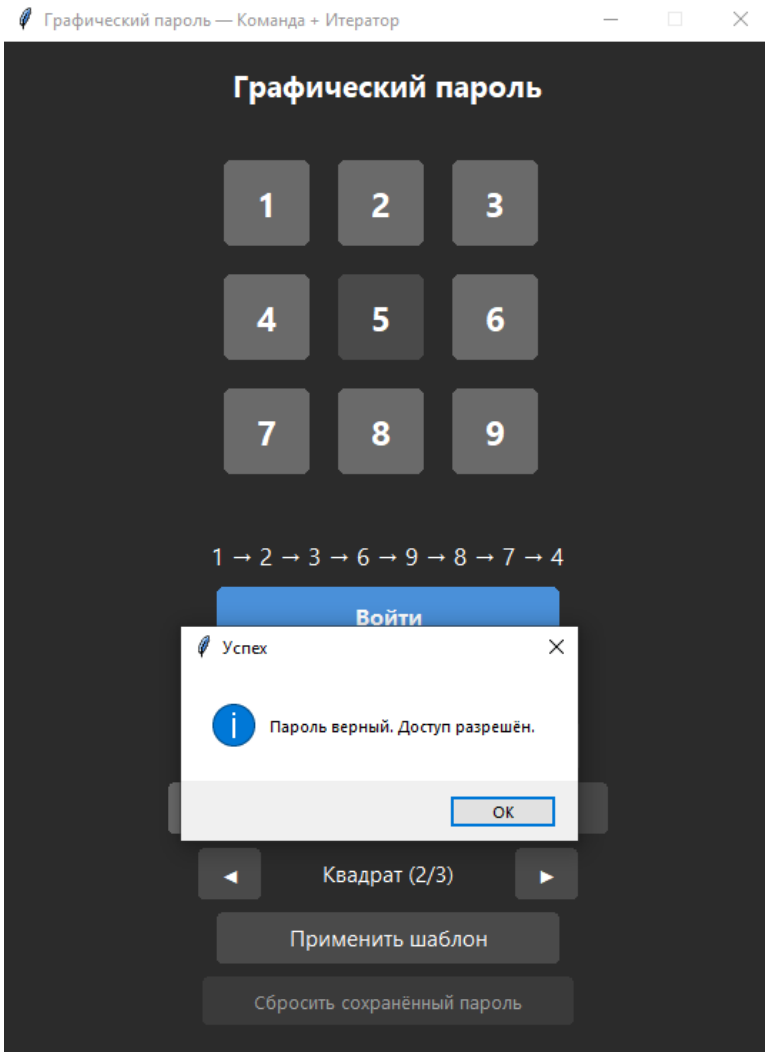


Рисунок 9 - Проверка правильного пароля

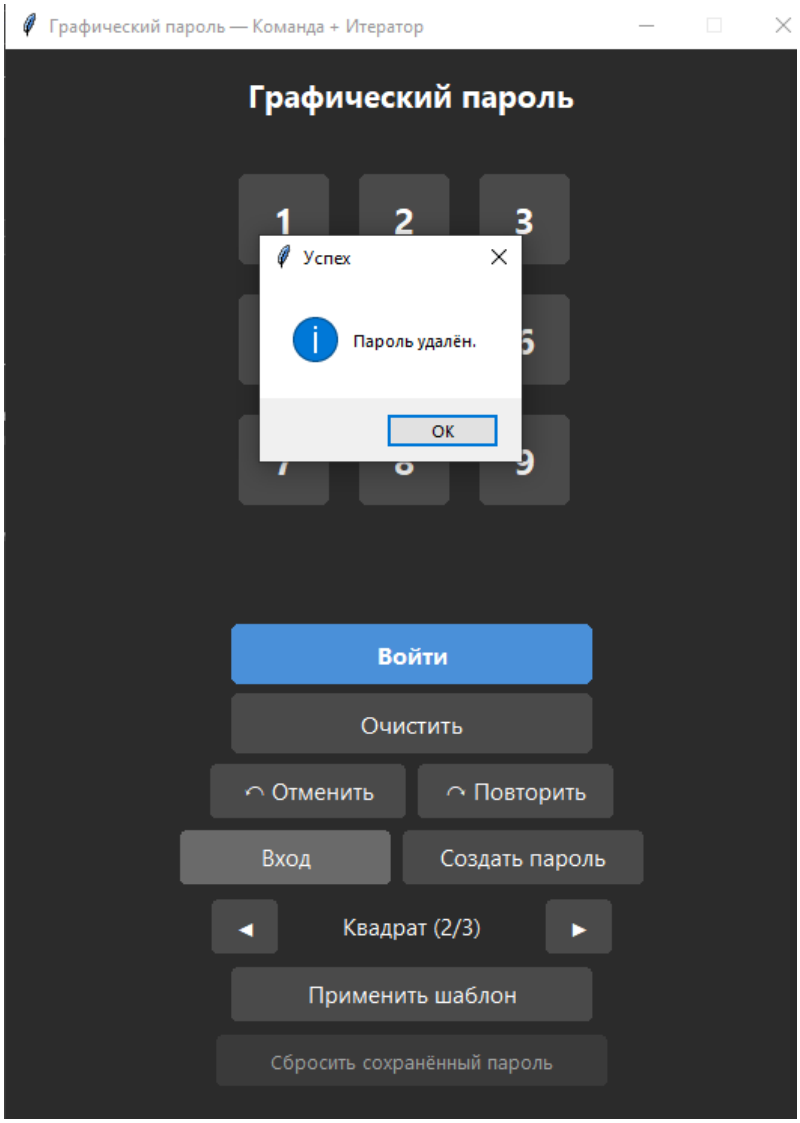


Рисунок 10 - Сброс сохранённого пароля

8. Исходный код

Основные элементы программы:

- Command, SelectCellCommand, ClearSelectionCommand, ApplyPatternCommand - команды;
- CommandInvoker - инициатор с историей и повторами;
- PatternIterator, PatternCollectionIterator, PatternCollection - итератор и агрегат;
- PasswordPattern - шаблон пароля;
- PasswordManager - Singleton с хэшированием;
- AppWindow - основной класс интерфейса;
- RoundedButton, rounded_rect() - скруглённые кнопки и ячейки;

– validate_sequence(), hash_sequence() - вспомогательные функции.

Полный исходный код доступен в репозитории GitHub: https://github.com/q1wf3/Patterny_Postnikov

Вывод

В ходе выполнения лабораторной работы было разработано приложение «Графический пароль» на языке Python с использованием библиотеки Tkinter. Были изучены и реализованы два поведенческих паттерна проектирования: «Команда» (Command) и «Итератор» (Iterator). Паттерн «Команда» применён для инкапсуляции действий пользователя (выбор ячейки, очистка, применение шаблона) с поддержкой отмены и повтора через CommandInvoker. Паттерн «Итератор» применён для последовательного обхода коллекции шаблонов через PatternIterator и PatternCollectionIterator. В результате получена работоспособная программа, демонстрирующая практическое применение поведенческих паттернов при разработке прикладного программного обеспечения.